

SQL SERVER / T-SQL

Programação de banco de dados

Revisão de conceitos

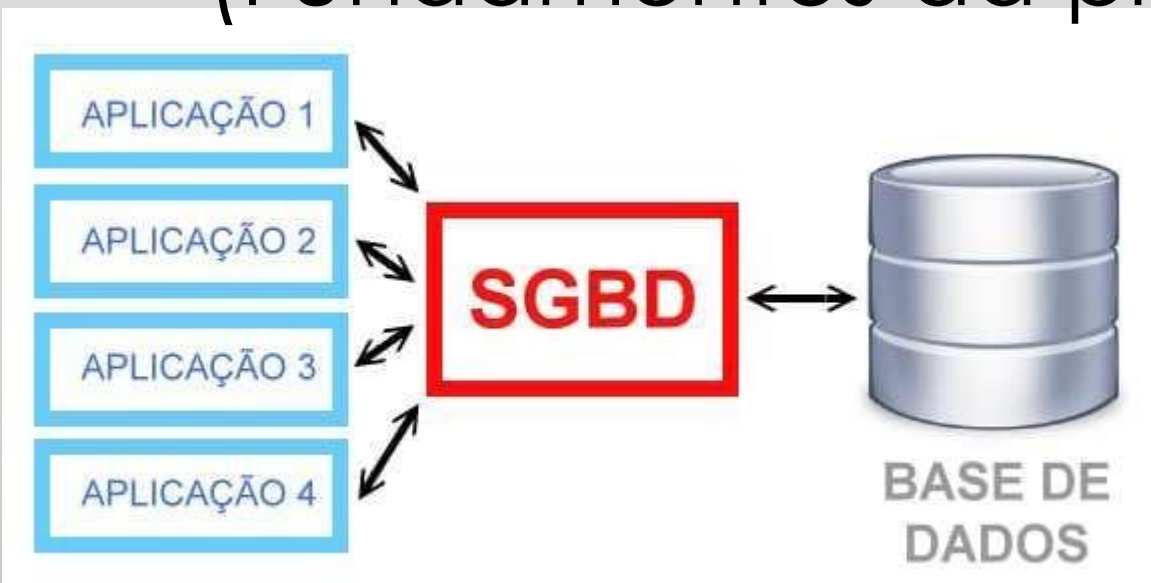
Prof. Me. Francisco Martins

LINKS ÚTEIS

1. Sqlserver Developer (<https://www.microsoft.com/pt-br/sql-server/sql-server-downloads>)
2. Sql Server Managment Studio (<https://docs.microsoft.com/pt-br/sql/ssms/download-sql-server-management-studio-ssms-19?view=sql-server-ver16>)
3. brModelo <https://sourceforge.net/projects/brmodelo/>

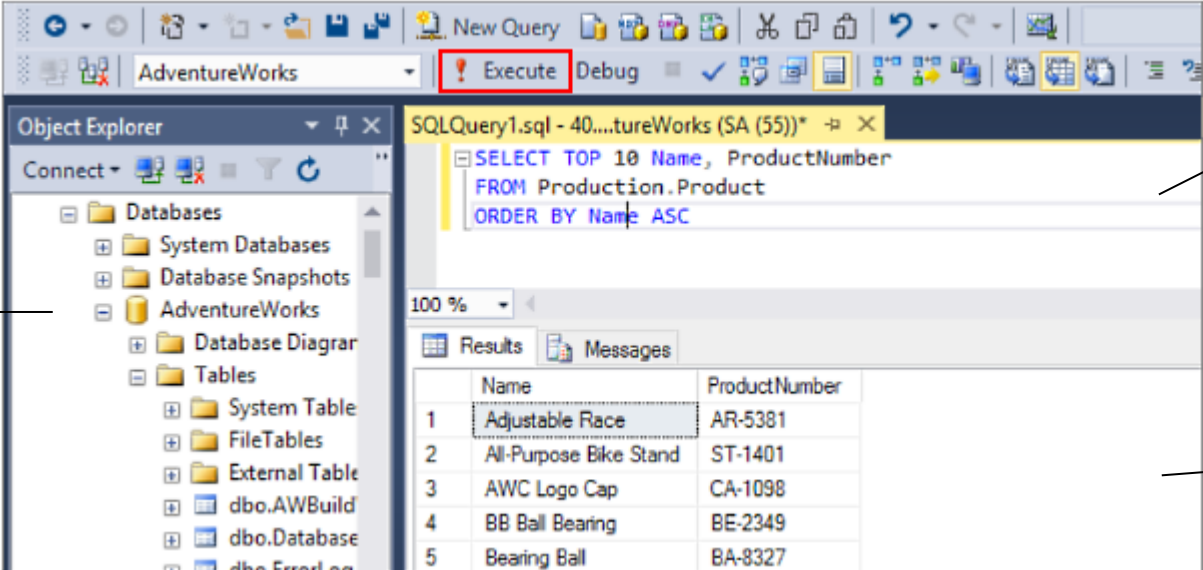
VISÃO GERAL DE SQL-SERVER E T-SQL

(Fundamentos da programação no SQL-Server)



SISTEMAS DE GERENCIAMENTO DE BANCO DE DADOS RELACIONAIS (SGBDR)

- Um SGBD é um sistema cujo objetivo principal é gerenciar o acesso e a correta manutenção dos dados armazenados em um ou mais bancos de dados. O acesso aos dados é disponibilizado por meio de uma interface que permite a comunicação com a aplicação desenvolvida.



The screenshot displays the Microsoft SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the database structure, with 'AdventureWorks' selected. The main window shows a query editor with the following SQL command:

```
SELECT TOP 10 Name, ProductNumber
FROM Production.Product
ORDER BY Name ASC
```

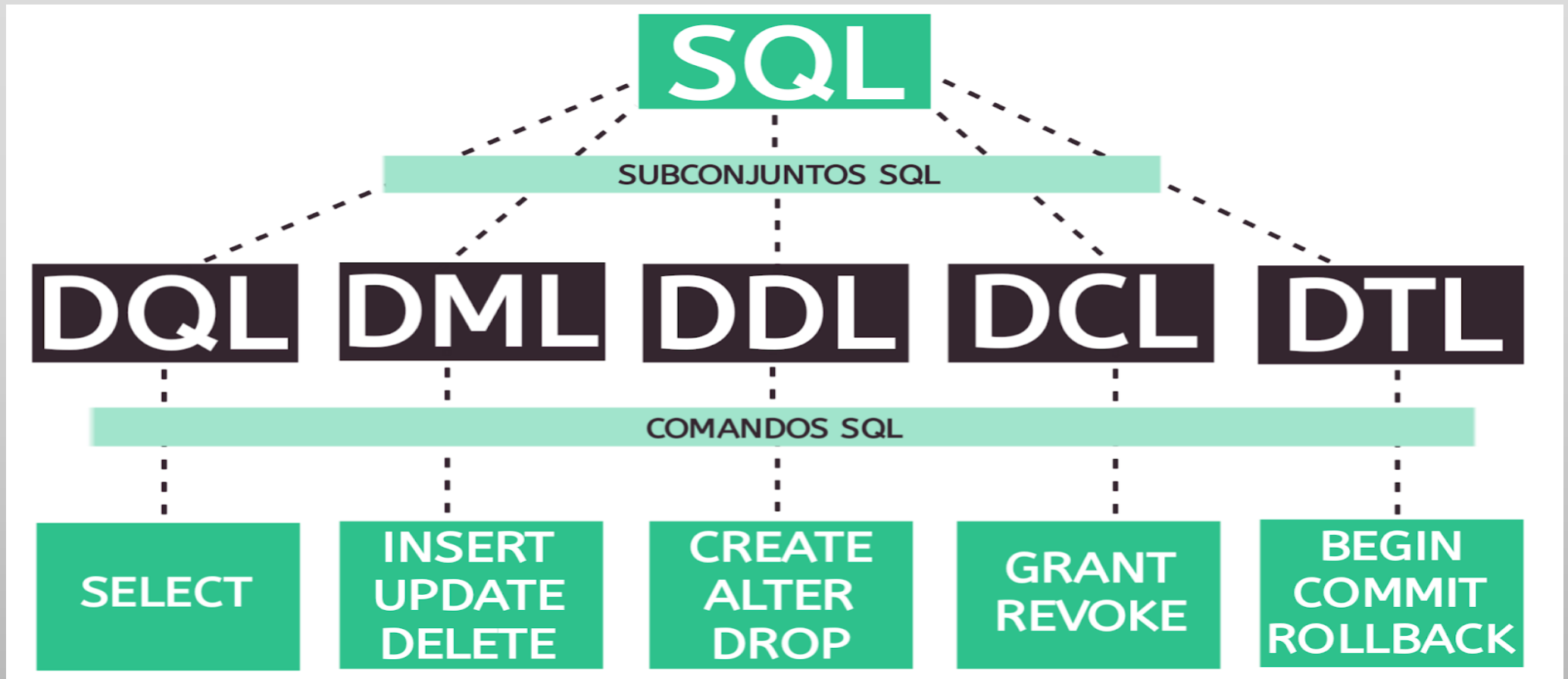
The 'Execute' button in the toolbar is highlighted with a red box. Below the query editor, the 'Results' pane shows the output of the query:

	Name	ProductNumber
1	Adjustable Race	AR-5381
2	All-Purpose Bike Stand	ST-1401
3	AWC Logo Cap	CA-1098
4	BB Ball Bearing	BE-2349
5	Bearing Ball	BA-8327

Annotations with arrows point to the 'Object Explorer' pane, the 'Execute' button, the SQL query text, and the 'Results' table.

<https://docs.microsoft.com/en-us/sql/linux/sql-server-linux-develop-use-ssms>

SUBCONJUNTOS DA LINGUAGEM SQL



SUBCONJUNTOS SQL E SEUS COMANDOS

- **DQL: linguagem de consulta de dados**
- DQL (Data Query Language) é o conjunto SQL que define o comando mais popular da linguagem, o **SELECT**. Esse comando é essencial para que possamos **consultar os dados que armazenamos em nosso banco**.

SUBCONJUNTOS SQL E SEUS COMANDOS

- **DML: linguagem de manipulação de dados**
- O DML (Data Manipulation Language) é o subconjunto do SQL que **define os comandos usados para manipular os dados armazenados em um banco**. Esse é um dos conjuntos mais utilizados, pois ele fornece operadores que nos permitem inserir, excluir e alterar os registros de uma tabela, por exemplo. Os comandos mais importantes desse subconjunto são: INSERT, DELETE e UPDATE.

SUBCONJUNTOS SQL E SEUS COMANDOS

- **DDL: linguagem de definição de dados**
- O DDL (Data Definition Language) é o subconjunto SQL que apresenta **comandos usados para gerenciar as estruturas do banco de dados**. Com ele, podemos criar, atualizar e remover objetos da base, como tabelas e índices. Os comandos definidos pelo DDL são: CREATE, DROP e ALTER.

SUBCONJUNTOS SQL E SEUS COMANDOS

- **DCL: linguagem de controle de dados**
- O DCL (Data Control Language) é o subconjunto no qual encontramos **comandos para controlar o acesso aos dados da nossa base**. Utilizando esse conjunto, conseguimos estabelecer restrições e permissões para quem acessa o banco por meio dos comandos GRANT(garantir) e REVOKE(revogar).

SUBCONJUNTOS SQL E SEUS COMANDOS

- **DTL ou TCL: linguagem de transação de dados**
- O DTL (Data Transaction Language) — também conhecido como TCL (Transaction Control Language) — é o subconjunto SQL que define comandos que utilizamos quando é necessário **gerenciar transações feitas no banco**. Isso significa que eles permitem iniciar, confirmar e desfazer determinadas alterações. Os comandos estabelecidos pelo conjunto são:
 - COMMIT(confirmar),
 - BEGIN(iniciair) e
 - ROLLBACK(descartar/desfazer).

COMMIT E ROLLBACK NO SQL SERVER

- Chamamos de Transação, em bancos de dados, a um conjunto de uma ou mais operações que constituem uma única tarefa ou unidade lógica de trabalho a ser processada.
- Os sistemas de bancos de dados devem garantir a execução correta das transações, mesmo que venham a ocorrer falhas; a transação deve ser executada por completo (incluindo todas as operações) ou então nenhuma de suas operações deve ser executada (a transação é abortada).
- **A transação é a propagação de alterações para o banco de dados, como por exemplo a alteração ou exclusão de registros em uma tabela.**

AS OPERAÇÕES COMMIT E ROLLBACK

- Se uma transação for concluída com sucesso (todas as operações bem-sucedidas), o banco de dados será alterado permanentemente, com os dados alterados persistidos (salvos em disco); essa operação é chamada de **COMMIT**.
- Porém, se houver falha em qualquer uma das operações que compõem a transação, o banco deverá ser retornado ao estado anterior ao início da transação; chamamos a essa operação de **ROLLBACK**.

PODE-SE CARACTERIZAR AS TRANSAÇÕES DA SEGUINTE FORMA:

- Todas as transações possuem início e fim
- As transações podem ser salvas (consolidadas no banco de dados) ou desfeitas
- Caso uma transação falhe no meio da execução de suas operações, nenhuma das operações será salva no banco de dados – todas serão desfeitas.
- No SQL Server, iniciamos uma transação com a declaração **BEGIN TRANSACTION**, e finalizamos o bloco que compõe a transação com as declarações **COMMIT TRANSACTION** ou **ROLLBACK TRANSACTION**, dependendo do estado que se deseja alcançar – salvar ou desfazer as operações, respectivamente.

EXEMPLO DE APLICAÇÃO

- No exemplo a seguir criamos uma tabela para testes de nome Livro_ISBN, com dados oriundos da tabela de livros. Logo após, experimentamos realizar duas transações: uma com rollback, desfazendo a transação, e outra com commit, confirmando a transação e alterando definitivamente os dados no banco.
- Vamos ao exemplo

■ criar tabela para testes

```
■ SELECT Nome_Livro,ISBN  
■ INTO Livro_ISBN  
■ FROM tbl_livros;
```

■ -- transação com rollback

```
■ BEGIN TRANSACTION  
■ DELETE FROM Livro_ISBN -- apaga todos registros da tabela, "sem querer"  
■ SELECT * FROM Livro_ISBN -- mostra tabela vazia  
■ ROLLBACK TRANSACTION; -- desfaz a transação  
■ SELECT * FROM Livro_ISBN; -- mostra os dados normalmente
```

■ -- transação com commit

```
■ BEGIN TRANSACTION  
■ DELETE FROM Livro_ISBN -- apaga todos registros da tabela  
■ SELECT * FROM Livro_ISBN -- mostra tabela vazia  
■ COMMIT TRANSACTION; -- confirma a transação  
■ SELECT * FROM Livro_ISBN; -- agora mostra a tabela vazia em definitivo
```

FUNÇÕES BÁSICAS DE UM SGBD

- **Restrições de Integridade (RI)**

- Integridade está associada à ideia de dados corretos, consistentes no banco de dados. As restrições de integridade preocupam-se em manter dados sempre coerentes, verdadeiros com realidade em questão.

Exemplo: uma turma de aluno só pode ter uma disciplina relacionada. *(No registro / na instância)*

FUNÇÕES BÁSICAS DE UM SGBDR

- Segurança
 - Esse controle evita a violação dos dados por agentes e/ou situações não previstas, ou seja, as falhas.
 - Políticas de autorização de acesso;
 - Recuperação de falhas.
- Controle de concorrência
 - Esse controle evita conflitos de acesso simultâneo a um dado por mais de uma transação.
- Independência dos dados
 - Significa transparência de gerenciamento e armazenamento, assim como do esquema global da organização, para as aplicações.

ARQUITETURA BÁSICA DE UM SGBDR

- Nível físico
 - Nível abstrato, em que se sabem **detalhes físicos** da organização de um dado, ou seja, o esquema físico.
 - **Dados estão armazenados em termos de bytes.**
- Nível conceitual
 - Nível intermediário, em que a preocupação está em descrever quais dados são significativos para uma organização, isto é, têm semântica, são relevantes para o dia a dia da organização.
 - Dados em nível de entidades/tabelas e relacionamentos.
 - **(Contexto ou Mini mundo).**

ARQUITETURA BÁSICA DE UM SGBDR

- Nível de visão
 - Nível mais alto de abstração, sendo particular de cada aplicação que manipula dados do banco de dados.
 - A aplicação determina o universo de dados que deseja ter acesso.

COMO OS DADOS SÃO ARMAZENADOS

- Diferença entre dado e informação
 - Podemos dizer que tudo o que é armazenado pode ser considerado como dado.
 - A informação surge quando agregamos dois ou mais dados e, a partir deles, e inferimos um conclusão.

Estado = São Paulo

Data = 16 de fevereiro de 2024

Nome = Maria da Penha

Conclusão possível com essa informação: ???

COMO OS DADOS SÃO ARMAZENADOS

- Dicionário de Dados (DD)
 - É o catálogo responsável pela manutenção dos metadados (são os dados dos dados) que dizem respeito à estrutura do esquema, à integridade, às configurações do SGBD para efeitos de controle, segurança e desempenho, as estimativas de acesso e estimativas sobre os dados.



SQL SERVER / T-SQL

Fundamentos de programação de banco de dados

Prof. Me. Francisco Martins

Cláusula CONSTRAINT

- Restrições são regras pré-definidas impostas às colunas de uma tabela.
- Usadas para controlar os dados que são inseridos, em nível de coluna ou de tabela.
- Podem ser especificadas no momento de criação da tabela (CREATE) ou após a tabela ter sido criada (ALTER)

CONSTRAINTS NO SQL

- NOT NULL
- UNIQUE
- CHECK
- PRIMARY KEY
- FOREIGN KEY
- DEFAULT



NOT NULL

- Por padrão, o SQL Server permite armazenamento de valores nulos em colunas das tabelas.
 - A constraint NOT NULL impede que uma coluna aceite valores nulos (NULL)
 - Deste modo, não é possível inserir um registro (ou atualizar) sem que haja valor neste campo.
-
- **Validade DATE NOT NULL**

UNIQUE

- Assegura que não haja valores duplicados em uma coluna ou combinação de colunas.
 - Pode haver várias constraints UNIQUE em uma tabela, mas apenas uma chave primária para cada tabela.
 - Uma constraint PRIMARY KEY automaticamente possui uma restrição UNIQUE definida.
-
- **NomeProduto VARCHAR(50) NOT NULL UNIQUE**

CHECK

- Define um número de valores válidos que podem ser inseridos em determinadas colunas.
 - Se for definida uma única coluna, ela permitirá apenas determinados valores para a coluna.
 - Usada para limitar os valores que podem ser colocados em uma coluna.
-
- **PrecoProduto MONEY NOT NULL CHECK(PrecioProduto > 0)**

PRIMARY KEY

- Uma chave primária identifica de forma única cada registro em uma tabela de banco de dados.
 - A coluna não pode conter valores repetidos e nem valores nulos (NULL).
 - Estabelece os relacionamentos entre tabelas.
 - Cada tabela só pode ter uma única chave primária.
-
- **CodProduto SMALLINT PRIMARY KEY**

DIFERENÇAS ENTRE PK E UNIQUE

Primary Key	Unique
Identifica cada linha de forma única em uma tabela.	Identifica valores únicos em uma coluna de uma tabela.
Não podem haver valores NULL.	Pode aceitar um valor NULL na coluna.
Uma tabela somente pode ter uma única chave primária.	Uma tabela pode ter mais de uma restrição unique.
Um índice clusterizado é criado por padrão.	Índices não-clusterizados são criados nas colunas.

FOREIGN KEY

- Cria relacionamentos entre tabelas, combinadas com chaves primárias (relacionamento pai-filho).
- Cada registro em uma coluna de chave estrangeira da tabela-filha deve possuir um registro correspondente na chave primária (tabela-pai).
- Não pode possuir valores nulos (NULL).
- Impõe integridade referencial às tabelas.

DEFAULT

- Fornece um valor padrão para uma coluna.
- O valor padrão é inserido em uma coluna, caso nenhum outro valor seja especificado durante a inserção de um novo registro.
- Usada para fornecer um valor padrão para uma coluna, quando nenhum valor é especificado.
- **CodFornecedor SMALLINT DEFAULT 1000**

EXEMPLO DE CONSTRAINT NA CRIAÇÃO DE TABELAS

- CREATE TABLE Produto (
 - CodProduto SMALLINT **PRIMARY KEY**,
 - NomeProduto VARCHAR(50) **NOT NULL UNIQUE**,
 - CodFornecedor **SMALLINT DEFAULT 1000**,
 - Validade **DATE NOT NULL**,
 - PrecoProduto **MONEY NOT NULL CHECK(PrecoProduto > 0)**,
 - **CONSTRAINT** fk_id_fornecedor **FOREIGN KEY** (CodFornecedor),
 - **REFERENCES** fornecedores (CodFornecedor) **ON DELETE CASCADE**);

Criar tabelas: CREATE TABLE

Sintaxe básica:

```
CREATE TABLE NomeTabela (  
    coluna1 tipo1 restrições,  
    coluna2 tipo2 restrições,  
    ...  
    colunaN tipoN restrições,  
    restrições adicionais  
);
```

BIBLIOTECA

- Como exemplo: será necessário criar um banco de dados para armazenar e gerir dados sobre livros, uma espécie de biblioteca particular.



Requisitos do Banco Biblioteca

Vamos armazenar os seguintes dados neste banco:

- Nomes dos Livros
- Nomes e Sobrenomes de Autores
- Nomes de Editoras
- Categorias de Assuntos
- Preços e Datas de Publicação dos Livros
- Número ISBN 13 de cada obra
- Quantidade de Páginas de cada livro

Requisitos do Banco Biblioteca

Também há algumas regras de negócio que seguiremos:

- Todo livro terá um nome e um ISBN cadastrados
- Todo autor terá nome e sobrenome cadastrado
- Os livros podem ter mais de um autor
- Códigos de ID dos livros iniciam a partir de 100
- Toda editora terá um nome cadastrado
- Todo livro pertencerá a um único assunto (principal)
- Todo livro terá preço, mas não é obrigatória a data de publicação
- O n° de páginas de cada livro é obrigatório

Tabelas do Banco Biblioteca

O banco será constituído por cinco tabelas relacionadas - quatro principais e uma associativa:

- Tabela de Livros
- Tabela de Autores
- Tabela de Editoras
- Tabela de Assuntos
- Tabela LivrosAutores (associativa)

DER DO BD DA BIBLIOTECA

- O DER do banco de dados poderá ser criado pelo SSMS após a correta criação das tabelas.
- Primeiro devemos criar as tabelas que não tem chaves estrangeiras (Regrinha).
- A cláusula IDENTITY só pode ser usada uma vez para cada tabela.

Principais Tipos de Dados

Tipo	Descrição
char(n)	String de caracteres de tamanho fixo n, máximo de 8000 caracteres.
varchar(n)	String de caracteres de tamanho variável n, máximo de 8000 caracteres.
tinyint	Números inteiros de 0 a 255
smallint	Números inteiros de -32768 a 32767
int	Números inteiros entre -2,147,483,648 e 2,147,483,647
money	Dados monetários de -922.337.203.685.477,5808 até 922.337.203.685.477,5807
real	Números de ponto flutuante de precisão dupla entre $-3,4 \times 10^{38}$ e $3,4 \times 10^{38}$
date	Data apenas. De 01/01/0001 a 31/12/9999
time	Hora apenas. Precisão de até 100 nanossegundos.

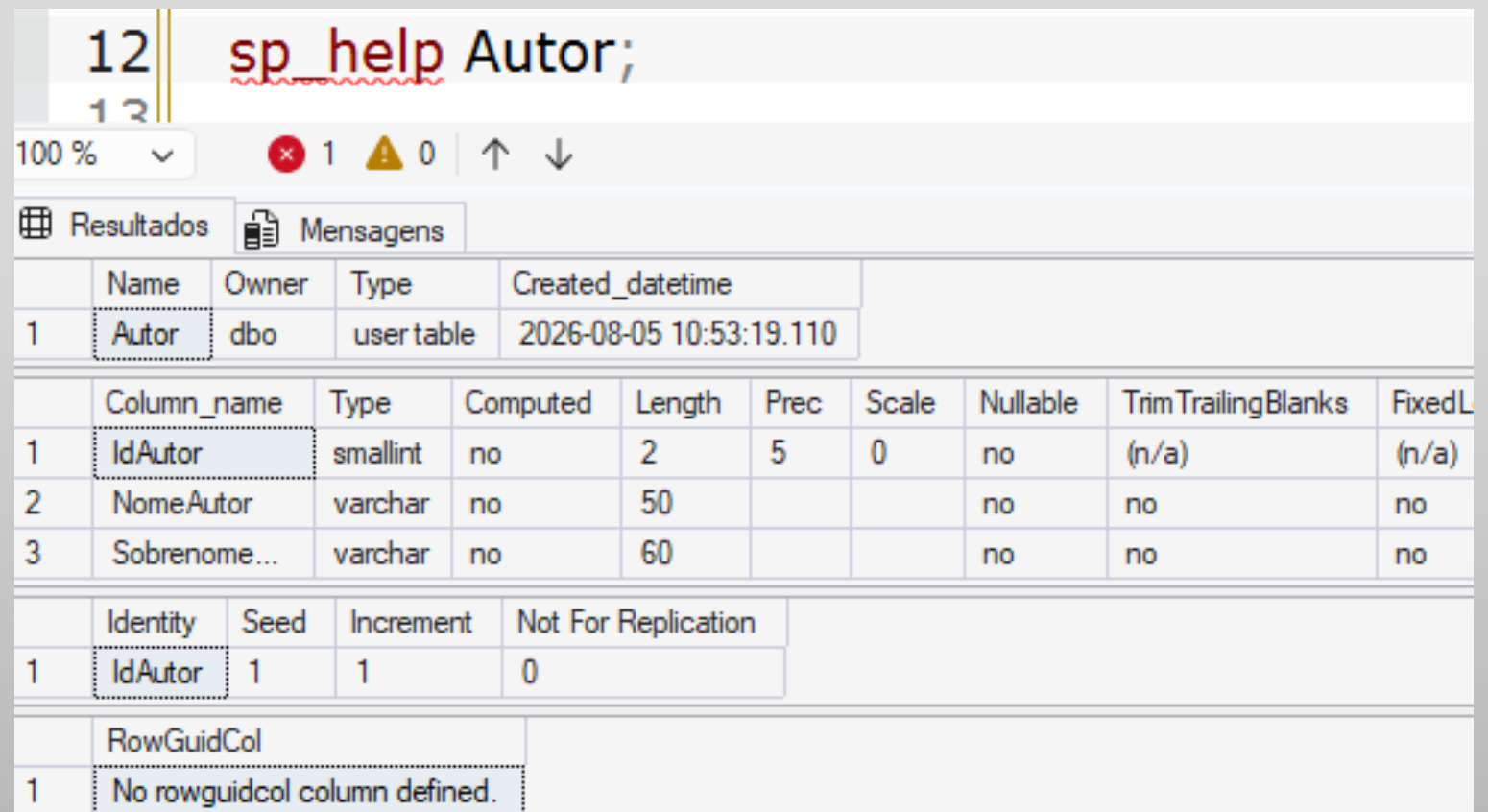
CRIANDO AS TABELAS DO BANCO DE DADOS BIBLIOTECA

- char / nchar = Caractere tamanho fixo / idem com Unicode
- varchar / nvarchar = Caractere tamanho variável / idem com Unicode
- Devemos dar um nome a toda constraint:
 - **pk_id_autor**

```
CREATE TABLE Autor (  
  IdAutor SMALLINT IDENTITY,  
  NomeAutor VARCHAR(50) NOT NULL,  
  SobrenomeAutor VARCHAR(60) NOT NULL,  
  CONSTRAINT pk_id_autor PRIMARY KEY(IdAutor)  
);
```

VISUALIZANDO DETALHES DA ESTRUTURA DA TABELA AUTOR

- O `sp_help` é um dos procedimentos armazenados que nos auxiliam com informações dos objetos do BD.
- **(sp=Stored Procedure)**



12 `sp_help Autor;`

100 % 1 0

Resultados Mensagens

	Name	Owner	Type	Created_datetime
1	Autor	dbo	user table	2026-08-05 10:53:19.110

	Column_name	Type	Computed	Length	Prec	Scale	Nullable	Trim TrailingBlanks	FixedL
1	IdAutor	smallint	no	2	5	0	no	(n/a)	(n/a)
2	NomeAutor	varchar	no	50			no	no	no
3	Sobrenome...	varchar	no	60			no	no	no

	Identity	Seed	Increment	Not For Replication
1	IdAutor	1	1	0

	RowGuidCol
1	No rowguidcol column defined.

CRIANDO AS TABELAS DO BANCO DE DADOS BIBLIOTECA

- Comandos SQL para criação das tabelas Editora e Assunto

```
CREATE TABLE Editora (  
  IdEditora SMALLINT PRIMARY KEY IDENTITY,  
  NomeEditora VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Assunto (  
  IdAssunto TINYINT PRIMARY KEY IDENTITY,  
  NomeAssunto VARCHAR(25) NOT NULL  
);
```

CRIANDO AS TABELAS DO BANCO DE DADOS BIBLIOTECA

```
CREATE TABLE Livro (  
  IdLivro SMALLINT NOT NULL PRIMARY KEY IDENTITY (100,1),  
  NomeLivro VARCHAR(80) not null,  
  ISBN13 char(13) unique not null,  
  DataPub date,  
  PrecoLivro money not null,  
  NrPaginas smallint NOT NULL,  
  IdEditora SMALLINT NOT NULL,  
  IdAssunto tinyint not null,  
  constraint fk_id_editora foreign key(IdEditora)  
    references Editora(IdEditora) on delete cascade,  
  constraint fk_id_assunto foreign key(IdAssunto)  
    references Assunto(IdAssunto) on delete cascade,  
  CONSTRAINT verifica_preco CHECK(PrecoLivro >= 0) );
```

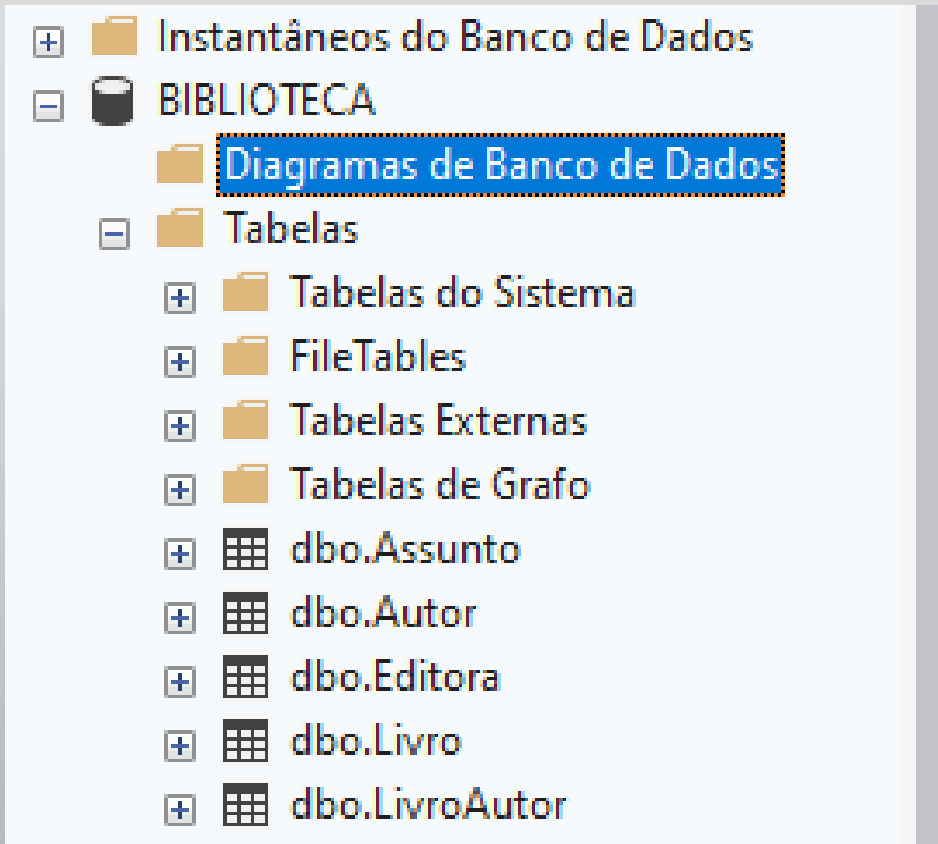
- Devemos observar que as restrições ou CONSTRAINTS da tabela Livro fazem referências às tabelas relacionadas de Editora e de Assunto, por isso, a tabela Livro deve ser criada após as tabelas “**Editora e Assunto**”.

CRIANDO AS TABELAS DO BANCO DE DADOS BIBLIOTECA

```
CREATE TABLE LivroAutor(  
  IdLivro SMALLINT NOT NULL,  
  IdAutor SMALLINT NOT NULL,  
  CONSTRAINT fk_id_livros foreign key(IdLivro) REFERENCES Livro(IdLivro),  
  CONSTRAINT fk_id_autores foreign key(IdAutor) REFERENCES Autor(IdAutor),  
  CONSTRAINT pk_livro_autor primary key(IdLivro, IdAutor)  
);
```

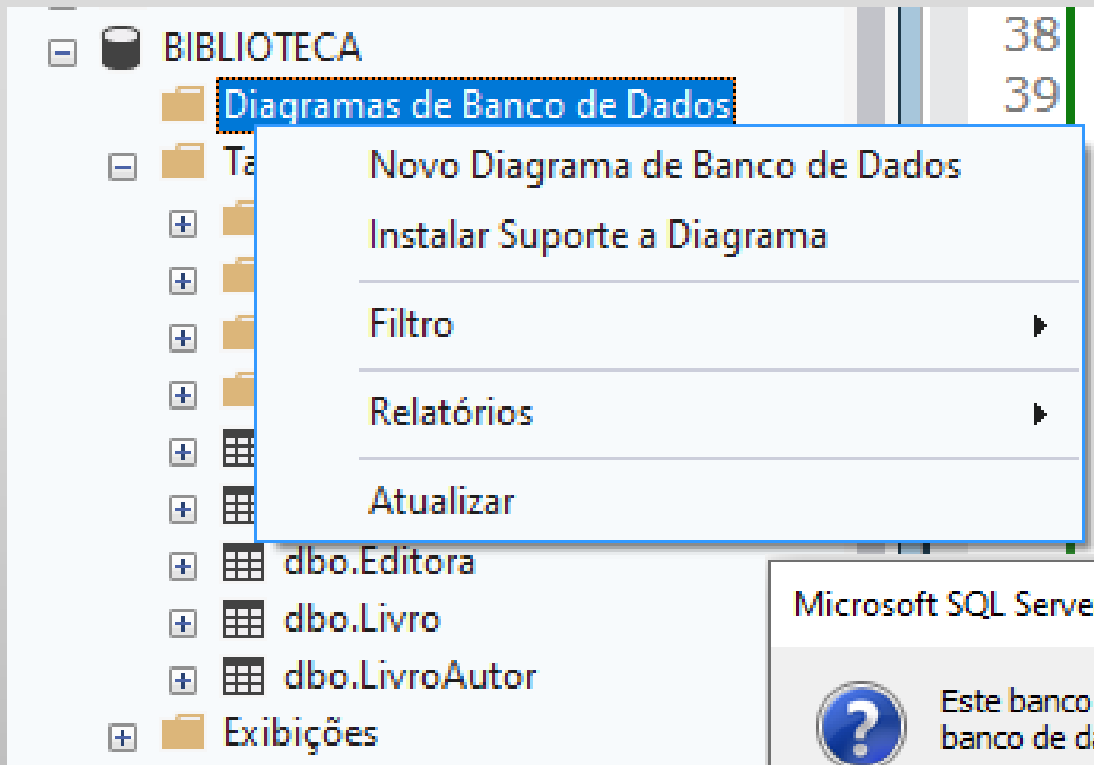
- Observe que as duas primeiras constraints referenciam as chaves estrangeiras das tabelas Livro e Autor e a terceira constraint cria a chave primária composta pelos atributos chaves primárias (PKs) das tabelas relacionadas.

MONTANDO O MER PELO SSMS

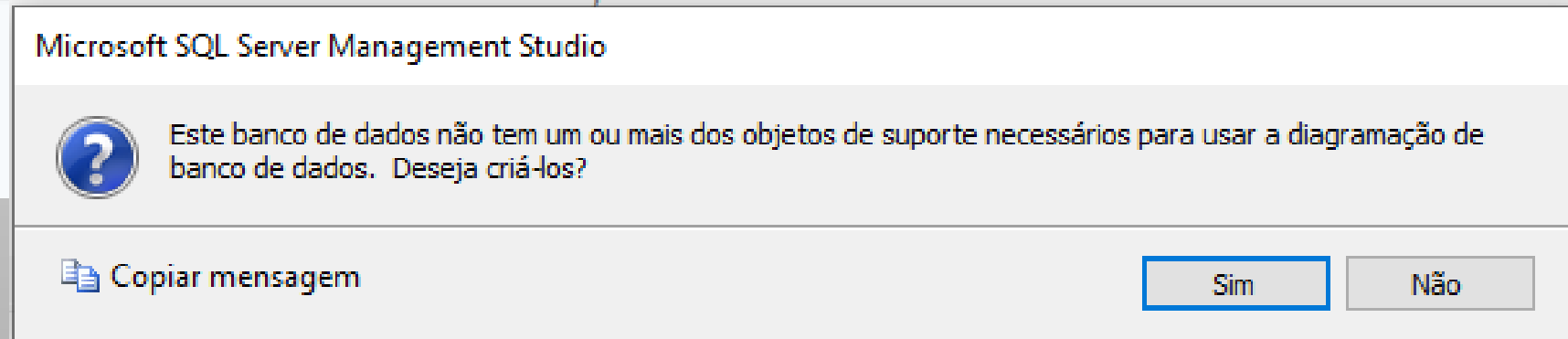


- Clique com o botão direito na janela à esquerda no IDE do SSMS (pesquisador de objetos), com o BD ativo na opção “Diagrama de Banco de Dados”.

MONTANDO O DER PELO SSMS



- Clique na janela à esquerda no IDE do SSMS (pesquisador de objetos), com o BD ativo na opção “Diagrama de Banco de Dados”.
- Em seguida no menu PopUp, clique em “Novo Diagrama de BD”.

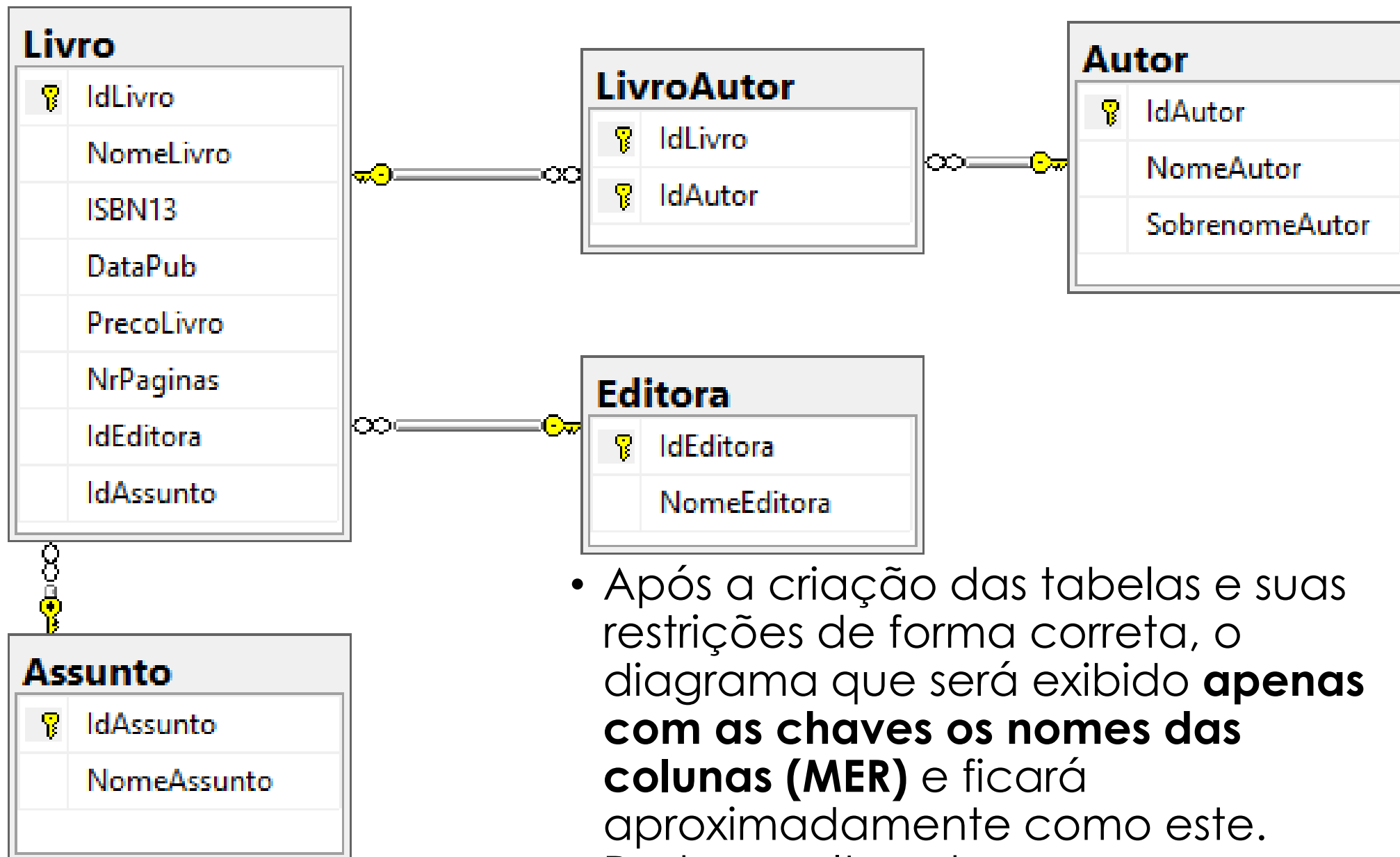


Adicionar Tabela ? X

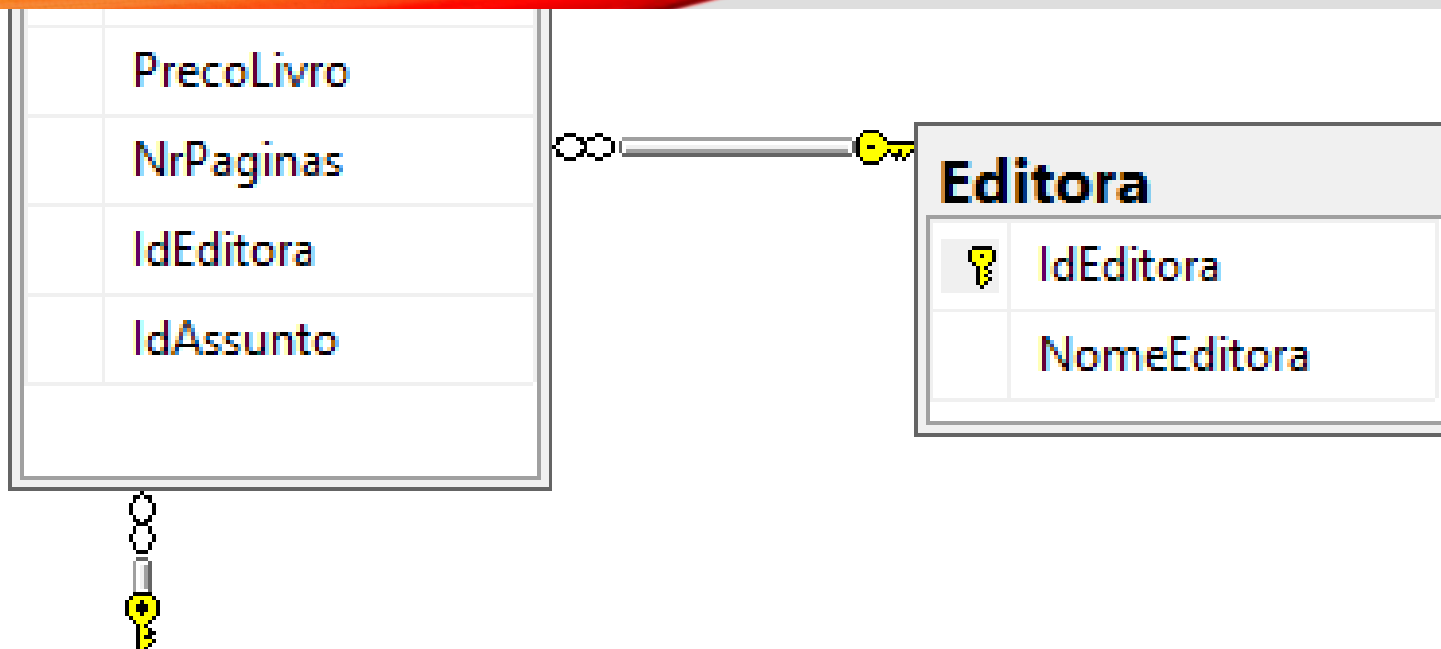
Tabelas

- Assunto
- Autor
- Editora
- Livro
- LivroAutor

Atualizar Adicionar Fechar



- Após a criação das tabelas e suas restrições de forma correta, o diagrama que será exibido **apenas com as chaves os nomes das colunas (MER)** e ficará aproximadamente como este. Pode ser alterado.



- O diagrama por padrão, exibe apenas **a chaves e os nomes das colunas**, mas poderá ser personalizado como a tabela Assunto abaixo: DER

Assunto					
	Nome da Coluna	Tipo de Dados	Comprimento	Permitir Nulos	Permite Valor Nulo
🔑	IdAssunto	tinyint	1	☐	Não
	NomeAssunto	varchar(25)	25	☐	Não
				☐	

SQL SERVER / T-SQL

Fundamentos de programação de
banco de dados

Exemplo

Criando e determinando a estrutura do BD

Prof. Me. Francisco Martins

DEFININDO A ESTRUTURA DO BD

```
1  ▾ -- Definições do arquivo de Banco de Dados
2    -- Será criado como primeira instância "ON PRIMARY"
3  ▾ CREATE DATABASE teste01 ON PRIMARY
4    (NAME= teste01,
5     FILENAME = 'C:\SQL\teste01.mdf', -- O caminho deve existir no disco
6     SIZE = 8MB,
7     MAXSIZE = 20MB, -- Crescimento até o tamanho especificado
8  ▾ FILEGROWTH = 10%) -- Cresce 10% do tamanho atual especificado
```

DEFININDO A ESTRUTURA LOG DO BD

```
10  -- Definições do arquivo de log
11  LOG ON (
12  NAME =teste01_log,
13  FILENAME = 'C:\SQL\teste01_log.ldf',
14  -- O caminho deve existir no disco
15  SIZE = 1MB,
16  MAXSIZE = 10MB,
17  FILEGROWTH = 1MB) -- Cresce em incrementos de 1MB
18  GO -- Garante que todos os comandos descritos
19  --até aqui sejam executados.
```

CONSULTAR BANCOS DE DADOS

-- Consulta à bds existentes - forma 1

SELECT name

FROM master.sys.databases

ORDER BY name; -- Lista os bds por ordem de nome

-- Existem Stored Procedures armazenados pelo sistema

EXEC sp_databases;

-- selecionar um banco para uso

USE teste01;

CONSULTAR BANCOS DE DADOS

```
-- Obter informações sobre um bd específico  
EXEC sp_helpdb teste01;  
  
-- Excluir banco de dados  
DROP DATABASE IF EXISTS TESTE02
```

CONSULTAR BANCOS DE DADOS

```
-- Excluir banco em uso  
USE master  
GO  
ALTER DATABASE teste01  
SET SINGLE USER WITH ROLLBACK IMMEDIATE  
DROP DATABASE teste01;
```

CONSULTAR BANCOS DE DADOS

```
-- Excluir banco em uso  
USE master  
GO  
ALTER DATABASE teste01  
--SET SINGLE USER WITH ROLLBACK IMMEDIATE  
SET SINGLE_USER WITH ROLLBACK IMMEDIATE  
DROP DATABASE teste01;
```

SQL SERVER / T-SQL

Fundamentos de programação de
banco de dados

Gerenciar Tabelas

Alter, Drop, Rename

Prof. Me. Francisco Martins

GERENCIANDO TABELAS

-- Gerenciamento de tabelas

-- Alter, Drop, Rename

-- ALTER TABLE NomeTabela

-- ADD / ALTER / DROP Objeto;

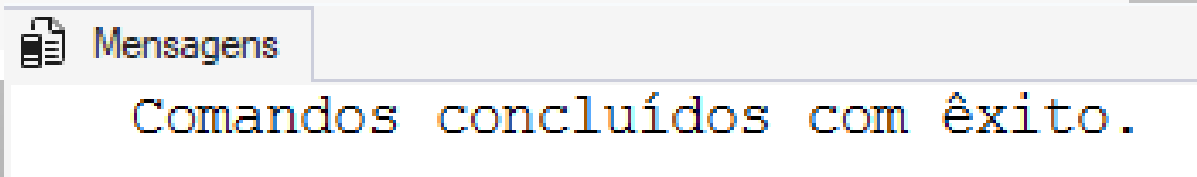
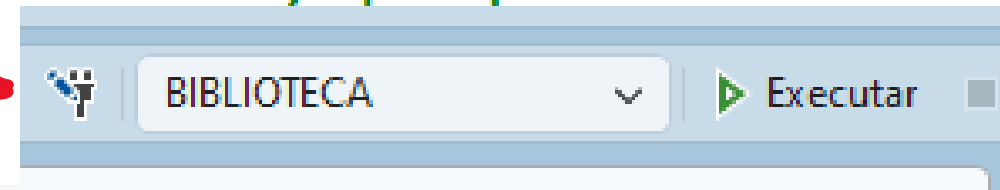
-- Adicionar uma nova coluna à uma tabela existente.

-- Devemos ativar o BD(Biblioteca) que possui a tabela.

USE BIBLIOTECA

ALTER TABLE Livro

ADD Edicao SMALLINT;



GERENCIANDO TABELAS

-- Alterar tipo de dados de uma coluna

```
ALTER TABLE Livro
```

```
ALTER COLUMN Edicao TINYINT;
```

-- Adicionar uma chave primária

```
ALTER TABLE Livro
```

```
ADD PRIMARY KEY (ISBN13);
```

GERENCIANDO TABELAS

-- Excluir uma constraint de uma coluna

```
ALTER TABLE NomeTabela
```

```
DROP CONSTRAINT NomeConstraint;
```

-- Verificar nomes das constraints:

```
sp_help Livro;
```

-- Excluir uma coluna de uma tabela

```
ALTER TABLE Livro
```

```
DROP COLUMN Edicao;
```

*Só poderá excluir tabela que não esteja relacionada (**restrições de dependência – PK / FK ou constraint de interligação**) com outras tabelas do Banco de Dados, e que não tenha índices e triggers.

GERENCIANDO TABELAS

```
-- Excluir uma tabela: DROP TABLE NomeTabela  
DROP TABLE Livro;
```

```
-- Renomear uma tabela: sp_rename  
-- sp_rename 'nome atual', 'Nome novo';  
sp_rename 'Livro', 'tbl_livros';
```

Mensagens

Cuidado: a alteração de qualquer parte de um nome de objeto pode interromper scripts e procedimentos armazenados.

*Só poderá excluir tabela que não esteja relacionada (**restrições de dependência – PK / FK ou constraint de interligação**) com outras tabelas do Banco de Dados, e que não tenha **índices e triggers**.

SQL SERVER / T-SQL

Fundamentos de programação de
banco de dados

Cadastrar Dados

INSERT INTO

Prof. Me. Francisco Martins

INSERIR DADOS

Inserir um registro (linha) em uma tabela -

Sintaxe:

```
INSERT INTO tabela (coluna1, coluna2,...)  
VALUES (valor1, valor2,...);
```

O BD que contém as tabelas deve estar ativo/exibido na janela de BD acima do pesquisador de objetos da IDE do SSMS.

*Primeiro inserir os dados nas tabelas que não dependem de outras tabelas.

INSERIR DADOS

```
-- Inserir Registros - O BD deve estar ATIVO na janela
-- Tabela de Assuntos
INSERT INTO Assunto (NomeAssunto)
VALUES
('Ficção Científica'), ('Botânica'), ('Eletrônica'), ('Matemática'),
('Aventura'), ('Romance'), ('Finanças'), ('Gastronomia'),
('Terror'), ('Administração'), ('Informática'), ('Suspense'),
('IA'), ('Ortopedia');
```

```
--Para resetar o valor de uma coluna IDENTITY para 0(Zero)
--DBCC CHECKIDENT ('NomeTabela', RESEED, 0);
DBCC CHECKIDENT ('Assunto', RESEED, 0);
```

INSERIR DADOS

```
select * from Assunto order by (IdAssunto);  
select * from Assunto order by (NomeAssunto);  
delete from Assunto;
```

	IdAssunto	NomeAssunto
1	1	Ficção Científica
2	2	Botânica
3	3	Eletrônica
4	4	Matemática
5	5	Aventura

- Para não ser necessário resetar, basta definir na criação da tabela:
- **IdAssunto TINYINT PRIMARY KEY IDENTITY(1,1)**
- “Coluna de preenchimento automático da tabela Assunto.

INSERIR DADOS

```
-- Tabela de Editoras
INSERT INTO Editora (NomeEditora)
VALUES
('Aleph'), ('Microsoft Press'), ('Wiley'), ('HarperCollins'),
('Érica'), ('Novatec'), ('McGraw-Hill'), ('Atlas'),
('Francisco Alves'), ('Sybex'), ('Globo'), ('Cia das Letras'),
('Morro Branco'), ('Record'), ('Martin Claret'), ('Penguin Books'),
('Springer'), ('Melhoramentos'), ('Ediouro'), ('Bookman');

select * from Editora order by (NomeEditora);
```

INSERIR DADOS

Próxima tabela será a de Autores...